CS633: Assignment 1

Group 26

Name	Roll No.	Program
Aarsh Jain	230015	BT-CSE
Bikramjeet Singh	230298	BT-CSE
Hitarth Kirankumar Makawana	230479	BT-CSE
Ronav Puri	230815	BT-CSE
Saksham Verma	230899	BT-CSE

Date of Submission: February 15, 2026

1 Code Description

1.1 Overview

The program `src.c` implements the data exchange pattern described in the assignment using the Message Passing Interface(MPI). Each process may send `M` double values to ranks located `D1` and `D2` positions to its right (in the logical topology), where `D1 < D2` depending upon whether those ranks are valid or not.

The global variables `myrank`, `size`, and `M` store the rank of the process, total number of processes, and data size respectively. If `myrank + D2 < size`, the process sends data to both `D1` and `D2`.If only `myrank + D1 < size`, it sends only to `D1`. Otherwise, it does not send data.

The data exchange, computation, and update are repeated for `T` iterations.

1.2 Data Buffers

Four buffers are allocated:

- `data_at_D1_og` – data to be sent to `D1`
- `data_at_D2_og` – data to be sent to `D2`
- `data_at_D1` – buffer to receive data from `D1`
- `data_at_D2` – buffer to receive data from `D2`

The "_og" buffers store the values that will be sent in the next iteration ($\therefore$ data treated to be as original/initial), while the other buffers store received values for computation.

1.3 Program Flow per Iteration

Each iteration consists of following stages:

1. Forward Communication

Forward communication is handled by the function:

```
Send_data(int d, double *send, double *recv, int tag)
```

It sends `send` to rank `myrank + d` and receives data from `myrank - d`, if valid.
It is called twice per iteration:

```
Send_data(D1, data_at_D1_og, data_at_D1, 10);
Send_data(D2, data_at_D2_og, data_at_D2, 20);
```

The tags here are different from any other communication to ensure safety.

2. Receiver Computation

Computation is performed inside:

```
Compute_data();
```

If `isRecvD1` is true, the received data is squared:

```
data_at_D1[i] = data_at_D1[i] * data_at_D1[i];
```

If `isRecvD2` is true, the logarithm is computed:

```
data_at_D2[i] = log(|data_at_D2[i]| + EPS)
```

Since there was no specific mention on how to handle negative values, we opted for taking the absolute values and adding a small positive value to ensure the positive domain for the log function.

This is because we were getting `NaN` values on account of negative values being passed to the logarithm function.

3. Return Communication

The computed data is sent back using:

```
Receive_data(int d, double *send, double *recv, int tag)
```

This is called as:

```
Receive_data(D1, data_at_D1, data_at_D1_og, 30);
Receive_data(D2, data_at_D2, data_at_D2_og, 40);
```

Here, the processed data replaces the original buffer for the next iteration.

4. Update Phase at Sender

After receiving computed values, updates are applied according to the assignment specifications:

For `D1`:

```
data_at_D1_og[k] = (unsigned long long) data_at_D1_og[k] mod 100000
```

For `D2`:

```
data_at_D2_og[k] = data_at_D2_og[k] * 100000.0
```

These updated buffers are used in the next iteration.

1.4 Communication Strategy

Only `MPI_Send` and `MPI_Recv` are used as specified in the assignment.

The overall communication strategy is inspired by the in-class example of 2-way 1D Nearest Neighbour communication where to improve performance, even and odd ranks were scheduled alternatively instead of scheduling right communications followed by the left ones.

This was done to remove the serialisation from the code resulting in a better performance and more non-interfering parallel communications.

To avoid serialisation with blocking communication, processes are divided into two groups using:

```
group = (myrank / d) mod 2
```

If `group == 0`, the process performs Send first and then Receive. Otherwise, it performs Receive first and then Send.
Separate MPI tags are used:

- 10 and 20 for forward communication
- 30 and 40 for return communication

Overall this communication strategy aims to complete as many parallel communications at the same time to improve the timings.

1.5 Initialization

Initial values are generated using:

```
srand(seed + myrank);
data_at_D1_og[i] = data_at_D2_og[i] = (double)rand() * (myrank + 1) / 10000.0;
```

The same initial values are used for both $D1$ and $D2$ buffers.

1.6 Timing Measurement

Timing is measured using:

```
double stime = MPI_Wtime();
...
double local_time = MPI_Wtime() - stime;
MPI_Reduce(&local_time, &total_time,1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
```

Overall every process calculates its own time taken. Using `MPI_Reduce()` the maximum time across all processes is calculated by rank 0 and reported in the final output.

1.7 Final Reduction and Output

Each valid sender computes the maximum value of data being communicated at distances `D1` & `D2` (whichever is valid):

```
maxD1 = max(data_at_D1_og)
maxD2 = max(data_at_D2_og)
```

Non-root processes send these values to rank 0 (only the valid sender) using tags 67 and 69.
Rank 0 receives all values, computes the global maxima, and prints:

```
printf("%lf %lf %lf\n", maxD1, maxD2, total_time);
```

The output format is:

$$\langle \texttt{maximumD1} \rangle \quad \langle \texttt{maximumD2} \rangle \quad \langle \texttt{time} \rangle$$

2 Code Compilation and Execution Instructions

2.1 Compilation

In order to separately compile the code, execute the following command in the terminal :

```
mpicc -o src.c src -lm
```

2.2 Execution

In order to run/execute the code for specific values of P, M, $D1$, $D2$, T and *seed*, execute the following command with custom inputs :

```
mpirun -np P ./src M D1 D2 T seed
```

However to run the job script, that follows the report experiments :

- $P = 8, 16, 32$
- $M = 262144, 1048576$
- $D1 = 2$
- $D2 = 4$
- $T = 10$
- seed = 1000

Each configuration is to be executed 5 times.
Execute the following bash files(On paramrudra CDAC server) :

- For $P = 8$, execute
  `./job8.sh`
- For $P = 16$, execute
  `./job16.sh`
- For $P = 32$, execute
  `./job32.sh`

The above mentioned scripts, specify the configurations (P, M, $D1$, $D2$, T, *seed*) and it also requires the user to compile the source file *src.c* using the command `mpicc -o src.c src -lm` before executing the job scripts. It creates a directory *raw_results* to store the *.tmp* output files. It executes *submit_job*() which runs five iterations, following the values of M, $D1$, $D2$, *seed* specified under configurations, where it runs the execution script `mpirun -np P ./src M D1 D2 T seed`. Finally it will store the results in a folder *raw_results* in *.tmp* format. Redirect the *.tmp* files output into a single *.csv* file named $final_results.csv$ using the script :

```
./redirect_tmp_to_csv.sh
```

which performs the following operations :

```
echo "P,M,maxD1,maxD2,Time" > final_results.csv

cat *.tmp >> ../final_results.csv

echo "Done!  Results saved in final_results.csv"
```

Basically adds a title to the final output file, redirecting the output *.tmp* files to a *.csv* file *final_results.csv*, and printing a success message.

3 Results

3.1 Performance Plot

The execution time was measured for 5 runs of each configuration. Figure 1 visualizes the data distribution, and Table 1 summarizes the key statistics.

Figure 1: Distribution of execution times over 5 runs for varying process counts (P) and data sizes (M). The left panel displays results for $M = 262,144$ and the right panel for $M = 1,048,576$.

Table 1: Summary of Execution Times and Slowdown Factors

Processes (P)	**Data Size (M)**	**Median Time (s)**	**Std. Dev (σ)**	**Slowdown (vs $P = 8$)**
Small Data Size ($M = 262,144$)				
8	262,144	0.089	0.002	-
16	262,144	0.345	0.040	3.88x
32	262,144	0.755	0.040	8.48x
Large Data Size ($M = 1,048,576$)				
8	1,048,576	0.370	0.004	-
16	1,048,576	0.720	0.020	1.95x
32	1,048,576	1.080	0.020	2.92x

3.2 Analysis of Results

- **Negative Scalability (Performance Decrease):**
  - **Observation:** The program does not get faster with more processes; instead, it gets significantly slower.
  - **Data:** Increasing processes from 8 to 32 caused execution time to rise by approximately **2.9x–8.5x**, depending on data size.
  - **Reason:** Since the workload per processor (M) is fixed, adding more processes only adds communication and synchronization overhead without reducing the computation load per core.
- **Stronger Impact on Smaller Data Sizes:**
  - **Observation:** The slowdown is significantly more severe for the smaller data size ($M = 262{,}144$).
  - **Data:** At $P = 32$, the small dataset is about **8.5x slower** than $P = 8$, whereas the large dataset is about **2.9x slower**.
  - **Reason:** When the computation workload per process is small, communication overhead dominates the total execution time. As the dataset size increases, the computation time grows proportionally, improving the computation-to-communication ratio. Consequently, the relative impact of communication overhead decreases, and we observe a better scaling.
- **The Two-Node Penalty ($P = 32$):**
  - **Observation:** $P = 32$ is consistently the slowest configuration.
  - **Context:** The assignment restricts us to 16 processes per node. Therefore, $P = 32$ forces the job to split across **two separate nodes**.
  - **Reason:** Communication between ranks on different nodes (Inter-node) travels over the network, which is significantly slower than the shared-memory communication (Intra-node) used when all processes are on one node ($P = 8, 16$).
- **Resource Contention ($P = 16$):**
  - **Observation:** The $P = 16$ configuration shows noticeable variance, especially for the smaller data size.
  - **Reason:** At $P = 16$, a single node is fully populated with all cores active. Processes compete for shared resources like memory bandwidth and cache, causing instability and occasional slower runs.

3.3 Code Efficiency

- **Algorithmic Efficiency: Linear Scaling with Respect to Data Size ($O(M)$)**
  - **Observation:** The code handles larger data sizes predictably.
  - **Data:** Quadrupling the data size ($4\times$) resulted in roughly a **4x** increase in execution time for $P = 8$, signaling linear growth.
  - **Conclusion:** This confirms the algorithm scales linearly with data size ($O(M)$ complexity).
- **Identified Inefficiency: Communication Overhead**
  - **Observation:** The runtime increases significantly as the number of processes grows, particularly for smaller data sizes.
  - **Interpretation:** This pattern indicates that communication overhead dominates execution time when computation per process is small.
  - **Likely Cause:** Since the implementation relies on blocking `MPI_Send` and `MPI_Recv`, processes must wait for communication to complete before proceeding. This waiting time likely contributes to the severe negative scaling observed at higher process counts.

4 Conclusions

In this assignment, we implemented a parallel data exchange program using MPI to analyze performance characteristics across varying process counts (P) and data sizes (M). The experimental results lead to the following key conclusions:

- **Severe Negative Scalability:** The application exhibits clear negative weak scaling behavior. As the number of processes increased from 8 to 32, execution time increased by up to **8.5x** for the smaller data size and nearly **3x** for the larger data size. This trend indicates that communication and synchronization overhead increasingly dominate the total runtime as more processes are added.
- **Impact of Hardware Topology:** The hardware constraint of 16 processes per node had a noticeable impact on performance. The $P = 32$ configuration was consistently the slowest, likely because it required inter-node communication over the network. This is inherently slower than the intra-node (shared memory) communication used when $P \leq 16$.
- **Algorithmic Efficiency:** Despite the poor parallel scaling, the program demonstrates linear growth with respect to data size. Quadrupling the input size resulted in approximately a $4\times$ increase in runtime (for $P = 8$), confirming $O(M)$ computational complexity and predictable behavior with increasing workload.

Contribution Statement: All group members contributed equally to the code development, debugging, and report generation.